Week 2 — Day 2: LangChain AI Agent

Scenario

The objective of this project was to rebuild the AI agent developed in the previous day using the LangChain framework. The previous agent was implemented using raw Python and Gemini API calls, where the complete agent workflow, tool execution, conversation history, and response handling were managed manually.

In this task, LangChain was used to provide a structured framework for developing the same type of intelligent agent. The agent was designed to understand user requests, select appropriate tools, execute those tools, use previous conversation information when required, and return useful responses.

The project focused on five tasks: understanding LangChain fundamentals, creating tools, building a tool-calling agent, implementing conversation memory, and adding structured output with error handling.

Introduction

Artificial Intelligence agents are applications that use Large Language Models to understand user requests and perform actions using external tools. A basic LLM can generate text, but an agent can go further by deciding when it needs additional information or when it should perform a specific operation.

In the previous implementation, the agent was developed using raw Python. This approach helped demonstrate the basic agent loop, but many components had to be handled manually. LangChain provides reusable abstractions that make it easier to connect language models with prompts, tools, memory, and structured outputs.

In this project, Google Gemini was connected with LangChain and used as the language model. Multiple tools were created, including a calculator, weather tool, and product-price tool. These tools allowed the agent to perform calculations, retrieve weather information, and obtain product prices.

The project also explored how an agent can maintain context between multiple conversations. A session-based memory system was implemented so that the agent could remember information from previous turns. Finally, Pydantic was used to create structured responses, while error handling was implemented to make the tools more reliable.

The overall purpose of this project was to gain practical experience in building a complete LangChain-based AI agent rather than simply sending individual prompts to an LLM.

Task 1 — LangChain Setup and Core Concepts

Objective

The objective of Task 1 was to understand the basic architecture of LangChain and set up the required environment for developing an AI agent.

Work Performed

The required LangChain packages were installed and configured. The Gemini model was connected with LangChain using ChatGoogleGenerativeAI.

The following important LangChain concepts were studied:

1. LLM

The Large Language Model is responsible for understanding user input and generating natural-language responses. Gemini was used as the LLM in this project.

2. Prompt Template

A prompt template provides a reusable structure for communicating instructions to the language model. Instead of writing the complete prompt every time, a template can be created and reused with different inputs.

3. Chains

A chain connects multiple components together so that the output of one component can become the input of another.

For example:

Prompt → LLM → Output

This creates a clear processing pipeline.

4. LCEL

LangChain Expression Language was studied for connecting components using the | operator.

The pipe operator means that the output from the component on the left is passed to the component on the right.

For example:

Prompt | LLM

means that the formatted prompt is passed directly to the language model.

5. Tools

Tools are functions that allow the AI agent to perform operations outside normal text generation. Examples include calculations, database lookups, APIs, and file searches.

Result

Task 1 established the basic LangChain environment and provided an understanding of the major components required for building an agent.

Task 2 — Creating and Integrating Tools

Objective

The objective of Task 2 was to create multiple tools and make them available to the LangChain agent.

Work Performed

Three tools were implemented using LangChain's @tool decorator.

1. Calculator Tool

A calculator tool was created to perform mathematical operations.

The tool supports operations such as:

- Addition
- Subtraction
- Multiplication
- Division

This allows the agent to delegate mathematical calculations to a dedicated tool.

For example, if the user asks the agent to calculate a 10% increase in a product price, the calculator can perform the required calculation.

2. Weather Tool

A weather tool was implemented to provide weather information based on a city.

The tool returns information such as:

- Temperature
- Weather condition

Example information can include:

Karachi → 34°C, Sunny

The agent can select this tool when the user asks about weather.

3. Product Price Tool

A product-price tool was implemented using a CSV file containing product information.

The tool searches the product data and returns the relevant product price.

For example:

Laptop A → 85,000 PKR

This demonstrates how an AI agent can interact with structured data through a custom tool.

Tool Integration

All three tools were registered with the agent:

Calculator + Weather + Product Price

The tools were then made available to the LangChain agent so that it could select the appropriate tool according to the user's request.

Result

Task 2 successfully demonstrated how normal Python functions can be converted into AI-agent tools using LangChain.

Task 3 — Tool-Calling Agent

Objective

The objective of Task 3 was to create an actual LangChain agent capable of deciding when to use a tool.

Work Performed

The LangChain create_agent() approach was used to create the agent according to the installed LangChain 1.4.0 environment.

The agent was provided with:

- Gemini LLM
- Calculator tool
- Weather tool
- Product-price tool

- System instructions

The agent was instructed to use the appropriate tool whenever the user's request required it.

Tool-Calling Test

The agent was tested with the request:

"What is the price of Laptop A?"

The agent identified that product information was required and called the product-price tool.

The tool returned:

Laptop A = 85,000 PKR

The agent then generated the final response:

The price of Laptop A is 85,000 PKR.

Multi-Step Calculation Test

The agent was also tested with:

Laptop A costs 85,000 PKR. Calculate its price after a 10% increase.

The agent calculated:

85,000 × 10% = 8,500

Then:

85,000 + 8,500 = 93,500 PKR

The final result was:

93,500 PKR

Execution Trace

The execution messages were inspected to understand the agent workflow.

The trace followed the pattern:

Human Message → AI Tool Call → Tool Message → AI Final Answer

This demonstrates the basic agent loop:

Reason → Act → Observe → Respond

Result

Task 3 successfully demonstrated a working LangChain tool-calling agent capable of selecting and using tools to solve user requests.

Task 4 — Conversation Memory

Objective

The objective of Task 4 was to make the agent capable of remembering information from previous conversation turns.

Work Performed

A session-based memory system was implemented using a memory store.

The memory stores both:

- User messages
- AI responses

A session ID was used so that each conversation could maintain its own history.

Turn 1

The user provided:

"Laptop A costs 85000 PKR."

The information was stored in the conversation history.

Turn 2

The user asked:

"What is its price after a 10% increase?"

The user did not repeat the laptop name or price.

The agent used the previous conversation context to understand that "its" referred to Laptop A and that the original price was 85,000 PKR.

The calculated result was:

93,500 PKR

Turn 3

The user asked:

"Would you recommend buying it?"

Again, the agent used the information from the previous turns to understand what product was being discussed.

Memory Verification

The stored conversation history was displayed to verify that the user and AI messages were successfully retained.

A final memory test was performed by asking:

"What was the original price of the laptop we discussed?"

The agent successfully recalled:

85,000 PKR

Result

Task 4 demonstrated that the agent could maintain conversational context and use previously provided information when answering follow-up questions.

Task 5 — Structured Output and Error Handling

Objective

The objective of Task 5 was to make the agent's responses more consistent and improve the reliability of tool execution.

Work Performed

1. Structured Output

A Pydantic model named ProductRecommendation was created.

The structured output contains:

- Product
- Price
- Currency
- Recommendation
- Reason

This ensures that the model produces information in a predictable structure instead of returning completely free-form text.

For example, the response can be represented as:

Product: Laptop A
Price: 85,000
Currency: PKR
Recommendation: Buy
Reason: Suitable based on the provided price and information.

2. Structured Output Testing

The structured model was tested with information about Laptop A.

The output was accessed through individual fields such as:

- product
- price
- currency
- recommendation
- reason

This demonstrated that the response could be processed programmatically.

3. Tool Failure Handling

Tool failures were also tested.

An invalid product such as:

Laptop Z

was provided to the product-price tool.

The tool execution was wrapped inside a try-except block so that the error could be captured and displayed rather than crashing the complete application.

4. Weather Error Handling

An unsupported city such as:

Multan

was also tested with the weather tool.

The error was captured using exception handling.

This demonstrated how invalid inputs can be handled safely.

5. LangChain vs Raw-Python

The LangChain implementation was compared with the previous raw-Python agent.

The raw-Python implementation required more manual handling of:

- Tool selection
- Agent workflow
- Conversation history
- Responses
- Error handling

LangChain provides reusable components that make these operations more organized and easier to manage.

Result

Task 5 successfully demonstrated structured responses and basic tool failure handling while showing the advantages of using LangChain over a manually implemented agent.

Overall Project Outcome

After completing all five tasks, a complete LangChain-based AI agent was developed.

The final system demonstrated:

- LLM integration
- Prompt and chain concepts
- Multiple custom tools
- Tool calling
- Multi-step reasoning
- Conversation memory
- Structured output
- Error handling
- Comparison with a raw-Python agent

The agent was successfully tested using product prices, calculations, weather information, and multi-turn conversations.

Overall Conclusion

This project successfully demonstrated the development of an AI agent using the LangChain framework. Starting from the basic LangChain concepts, the project progressed toward a complete agent capable of using multiple tools, performing calculations, retrieving product information, maintaining conversation context, and generating structured responses.

The implementation showed that LangChain provides a more organized approach to agent development compared with building the complete workflow manually with raw Python. Tools can be registered and reused easily, while agent execution provides a structured way for the language model to decide when a tool is required.

The memory implementation demonstrated that AI agents can maintain context across multiple conversation turns, allowing users to ask follow-up questions without repeating previous information. Structured output further improved consistency by forcing responses into a predefined format, while error handling made the tools more robust when invalid inputs were provided.

Overall, the project provided practical knowledge of **LLMs, prompts, LCEL, tools, agents, tool calling, memory, structured output, and error handling**. These concepts form important foundations for developing more advanced and reliable AI-agent applications using modern frameworks.